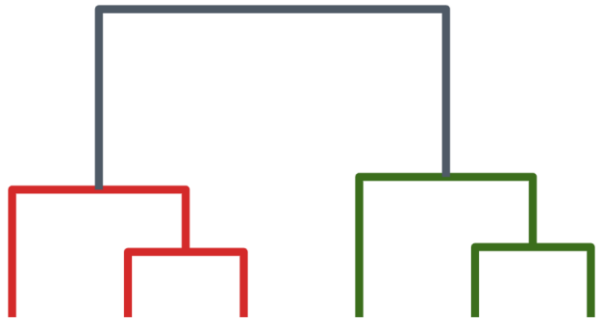
Hierarchical Clustering

Bottom-up (agglomerative) · Top-down (recursive k-means)

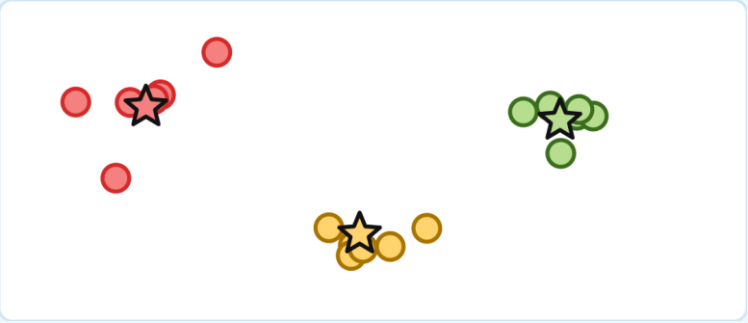


[Presenter name] · [Course] · [Date]

From Flat to Nested Clusters

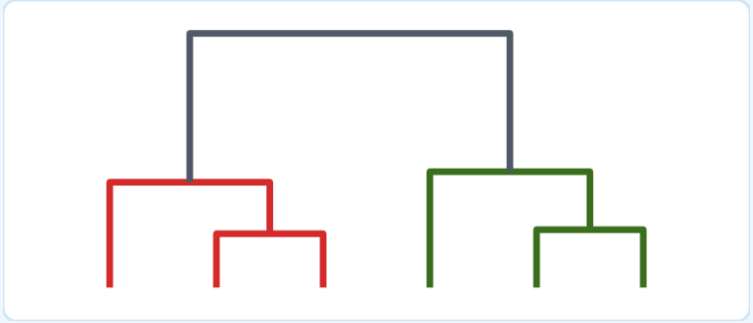
Partitioning gives one level; hierarchical clustering gives all levels

Partitioning



- One flat set of k clusters
- k is chosen before running
- k-means, k-means++, k-medoids

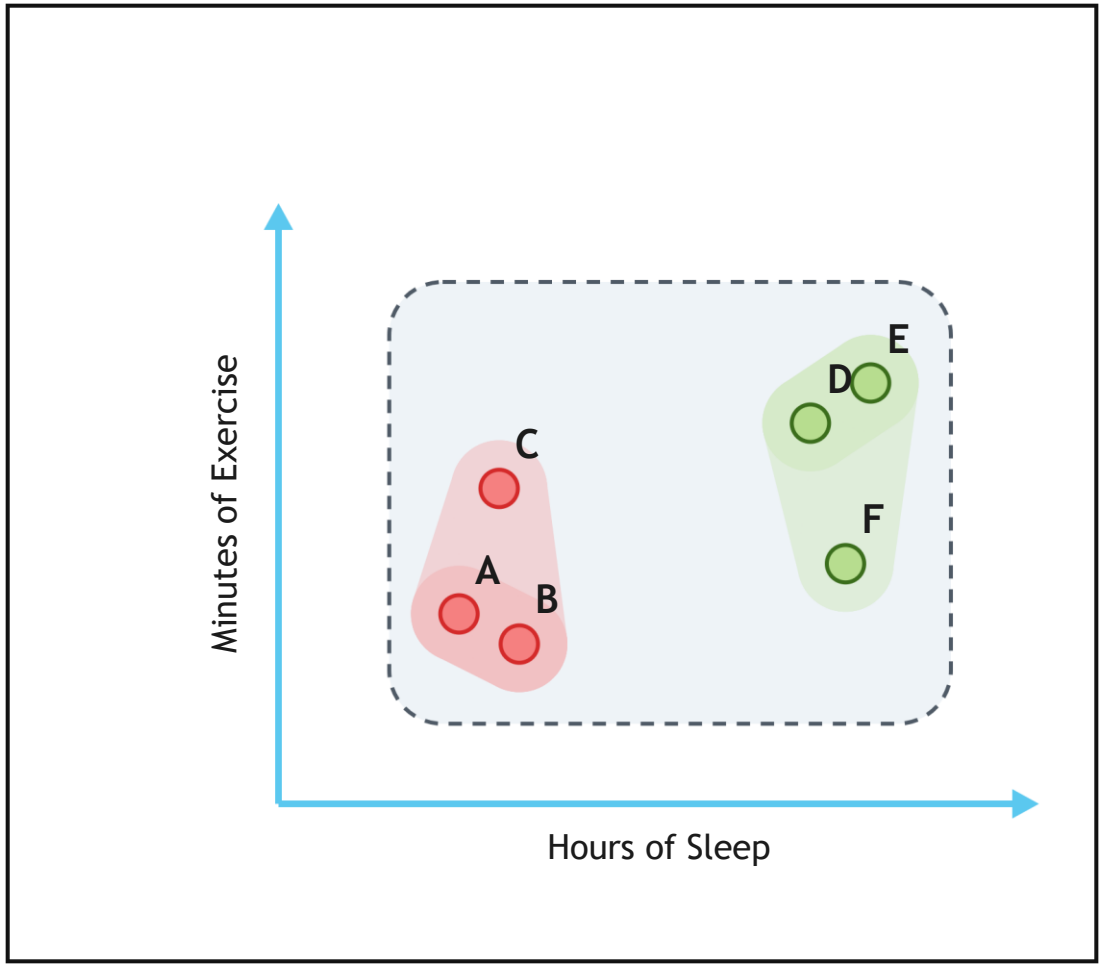
Hierarchical



- A tree of nested clusters
- Choose the level afterwards
- Agglomerative, divisive

What Is Hierarchical Clustering?

Clusters inside clusters, organised as a tree



Hierarchy

A tree of nested clusters, from single objects up to one cluster holding everything.

Dendrogram

The tree diagram that records the order and distance of every merge or split.

No k up front

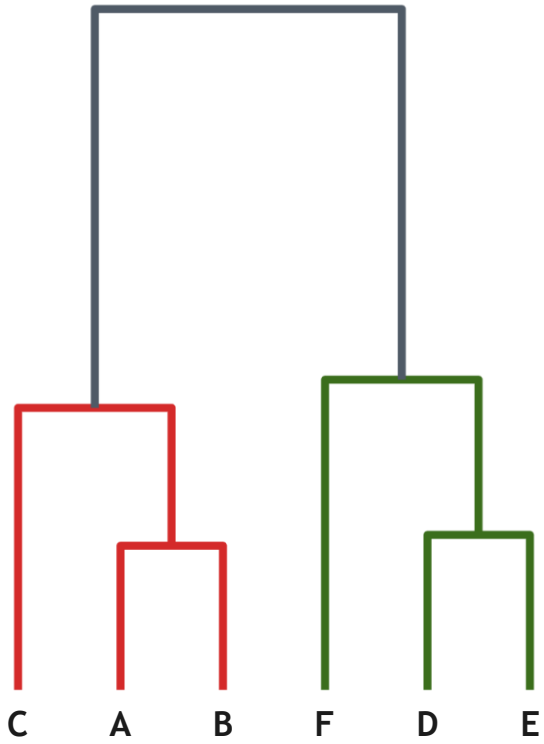
Pick the number of clusters afterwards by cutting the tree.

Two Ways to Build the Tree

Merge from the bottom, or split from the top

Bottom-up Agglomerative

Start with 6 single points. Merge the two closest clusters, again and again.

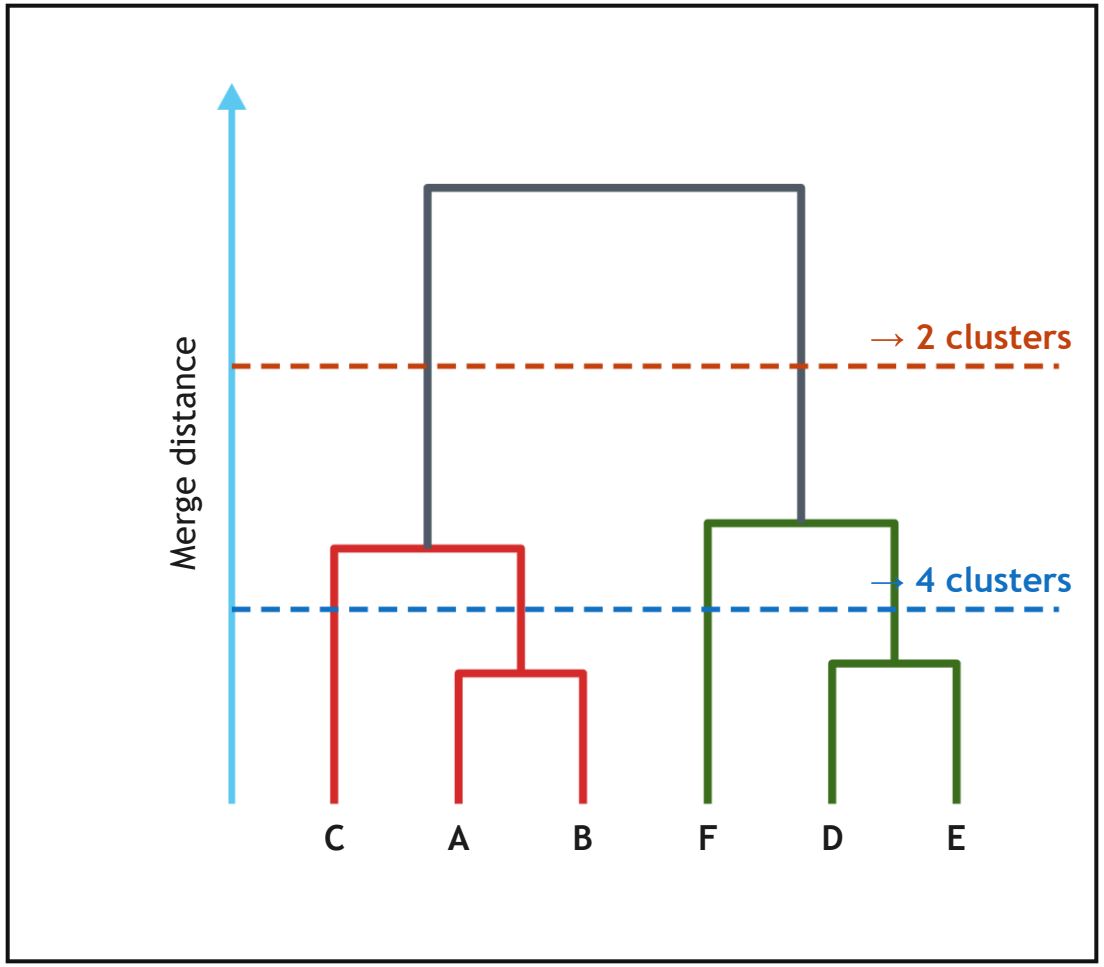


Top-down Divisive

Start with 1 cluster of everything. Split a cluster in two, again and again.

Reading a Dendrogram

Height shows how far apart two clusters were when they joined



Leaves

The bottom row: one leaf per object.

Height

Where two branches join = the distance at which they merged.

Cut

A horizontal cut gives clusters: count the branches it crosses.

Part 1 of 2

Bottom-Up

Agglomerative clustering

The Agglomerative Algorithm

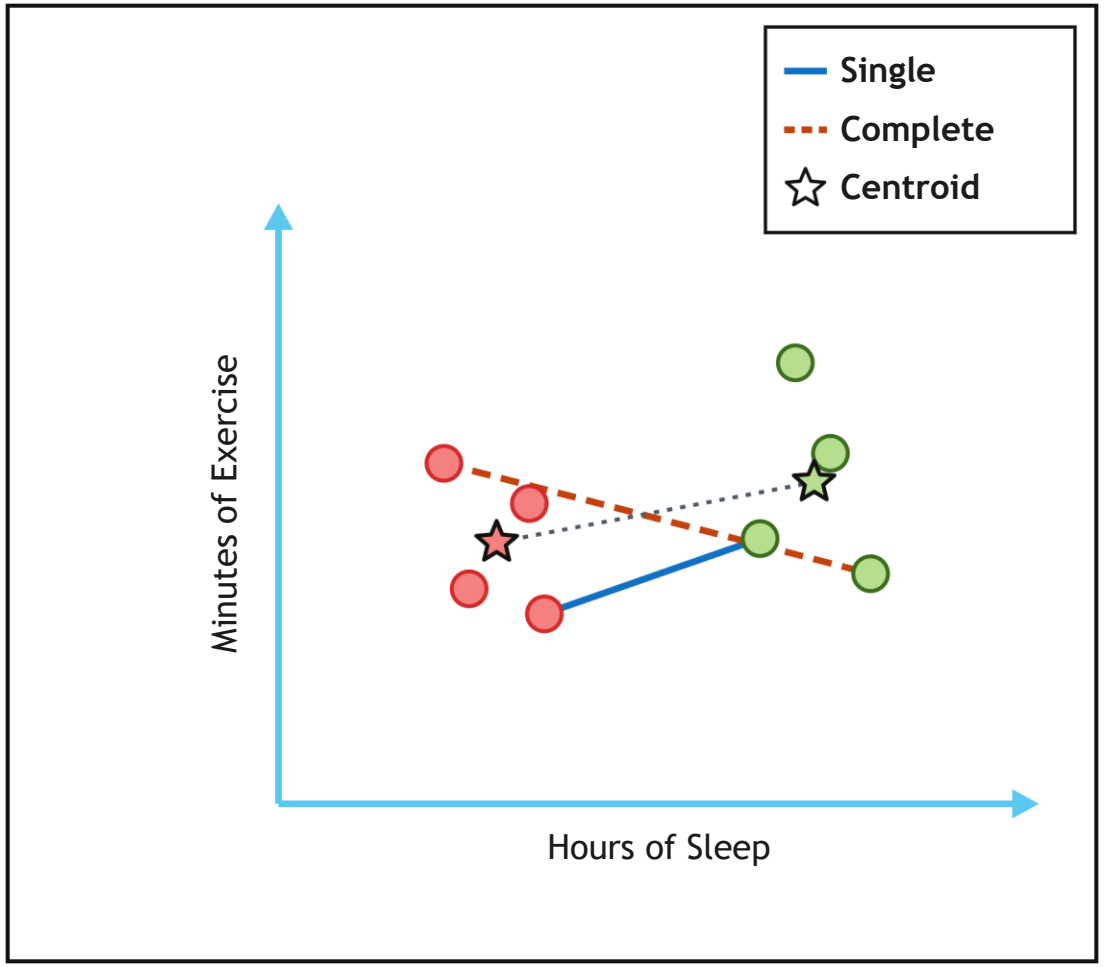
Start with n clusters, merge until one is left

1	**Start**	Put every object in its own cluster: n clusters.
2	**Distances**	Compute the distance between every pair of clusters.
3	**Merge**	Merge the two closest clusters into one.
4	**Update**	Recompute distances from the new cluster to all others.
5	**Repeat**	Repeat steps 3–4 until one cluster remains (or k remain).

Each merge is recorded in the dendrogram, at the height of the merge distance.

How Far Apart Are Two Clusters?

The linkage criterion defines distance between clusters



Single link
Distance between the closest pair of points.

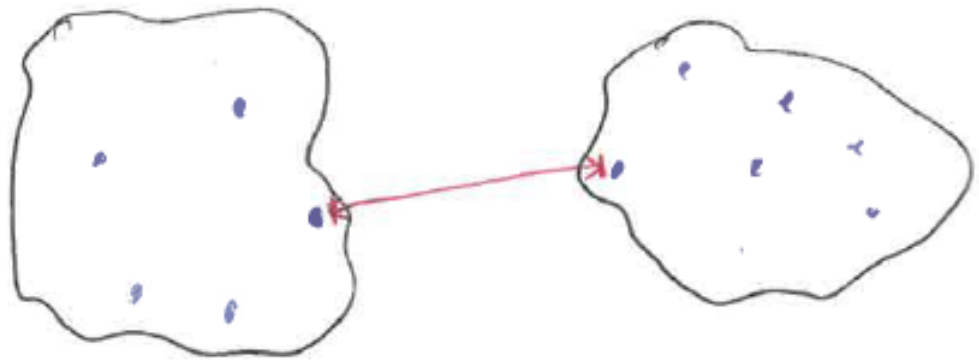
Complete link
Distance between the farthest pair of points.

Average link
Average distance over all pairs of points.

Ward
Merge the pair that least increases total SSE.

MIN or Single link (Nearest Neighbor Clustering Algorithm)

- The distance between two clusters is represented by the distance of the closest pair of data objects belonging to different clusters.
- Determined by one pair of points i.e., by one link in the proximity graph.



$$d_{min}(C_i, C_j) = \min_{p \in C_i,\ q \in C_j} d(p, q)$$

$$\min \{ d(P_1, q_1), d(P_1, q_2) \dots d(P_1, q_n), d(P_2, q_1), d(P_2, q_2) \dots$$
$$d(P_2, q_n) \dots d(P_m, q_1), d(P_m, q_2) \dots d(P_m, q_n) \}$$

- Can handle non-elliptical shapes (adv)
- Sensitive to noise & outliers (dis adv)

MAX or Complete link (Farthest-Neighbor Clustering Algorithm)

The distance between two clusters is represented by the distance of the farthest pair of data objects belonging to different clusters.



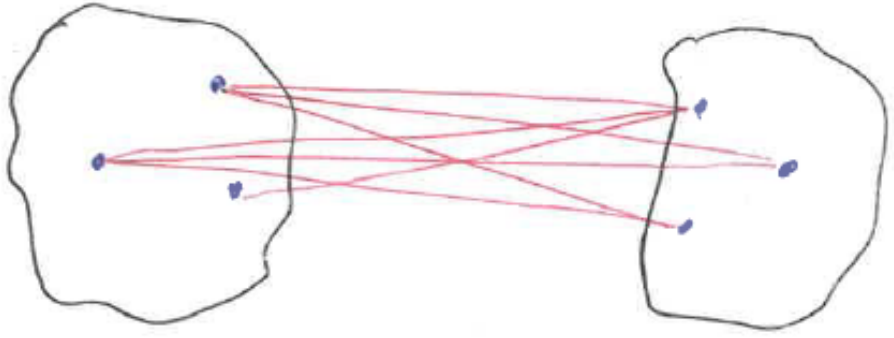
$$d_{min}(C_i, C_j) = \max_{p \in C_i,\, q \in C_j} d(p, q)$$

- Less susceptible to noise and outliers (adv)
- Tends to break large clusters (dis adv)
- Biased towards globular clusters (dis adv)

- Group Average or Average Link

The distance between two clusters is represented by the average distance of all pairs of data objects belonging to different clusters.

- Determined by all pairs of points in the two clusters



$$d_{min}(C_i, C_j) = \underset{p \in C_i,\ q \in C_j}{avg}\ d(p, q)$$

Compromise between Singh & complete link

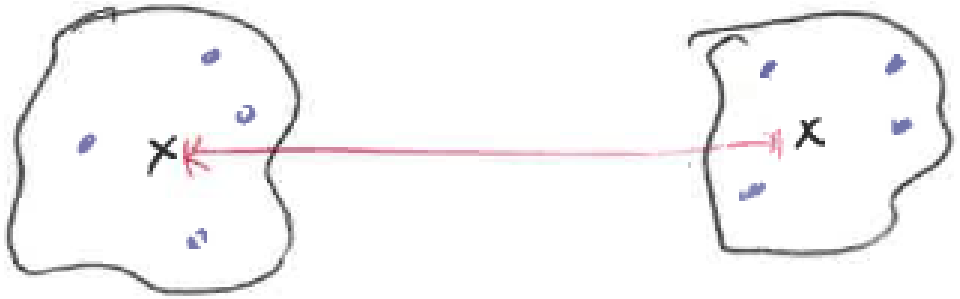
Less susceptible to noise & outliers (adv)

Biased towards globular clusters (dis adv)

4. Centroid Distance

- The distance between two clusters is represented by the distance between the centers of the clusters

- Determined by cluster centroids



$$d_{mean}(C_i, C_j) = d(m_i, m_j)$$

5. Ward's Method:

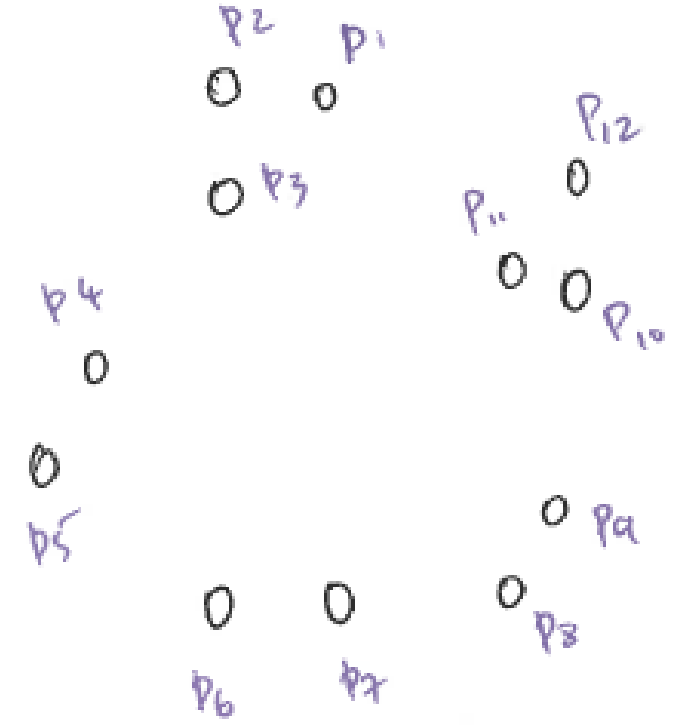
- Similarity of two clusters is based on the increase in squared error when two clusters are merged
- Similar to group average if distance between points is distance squared
- Less susceptible to noise and outliers (adv)
- Biased towards globular clusters (dis adv)

Agglomerative in Action

1. Updating the distance matrix

START SITUATION

Start with clusters of individual points & a distance matrix



Distance Matrix

	p_1	p_2	p_3	...	p_{12}
p_1	0	d_{12}	d_{13}		$d_{1\,12}$
p_2	d_{21}	0			
p_3	d_{31}		0		
p_4	d_{41}				
⋮					
p_{12}	$d_{12\,1}$				0

Agglomerative in Action

INTERMEDIATE SITUATION

After some merging steps, we have some clusters



Distance Matrix

	C_1	C_2	C_3	C_4	C_5
C_1	0	d'_{12}	d'_{13}		
C_2	d'_{21}	0			
C_3	d'_{31}		0		
C_4				0	
C_5					0

We want to merge two closest clusters & update the distance matrix

(C_2, C_5)

After merging

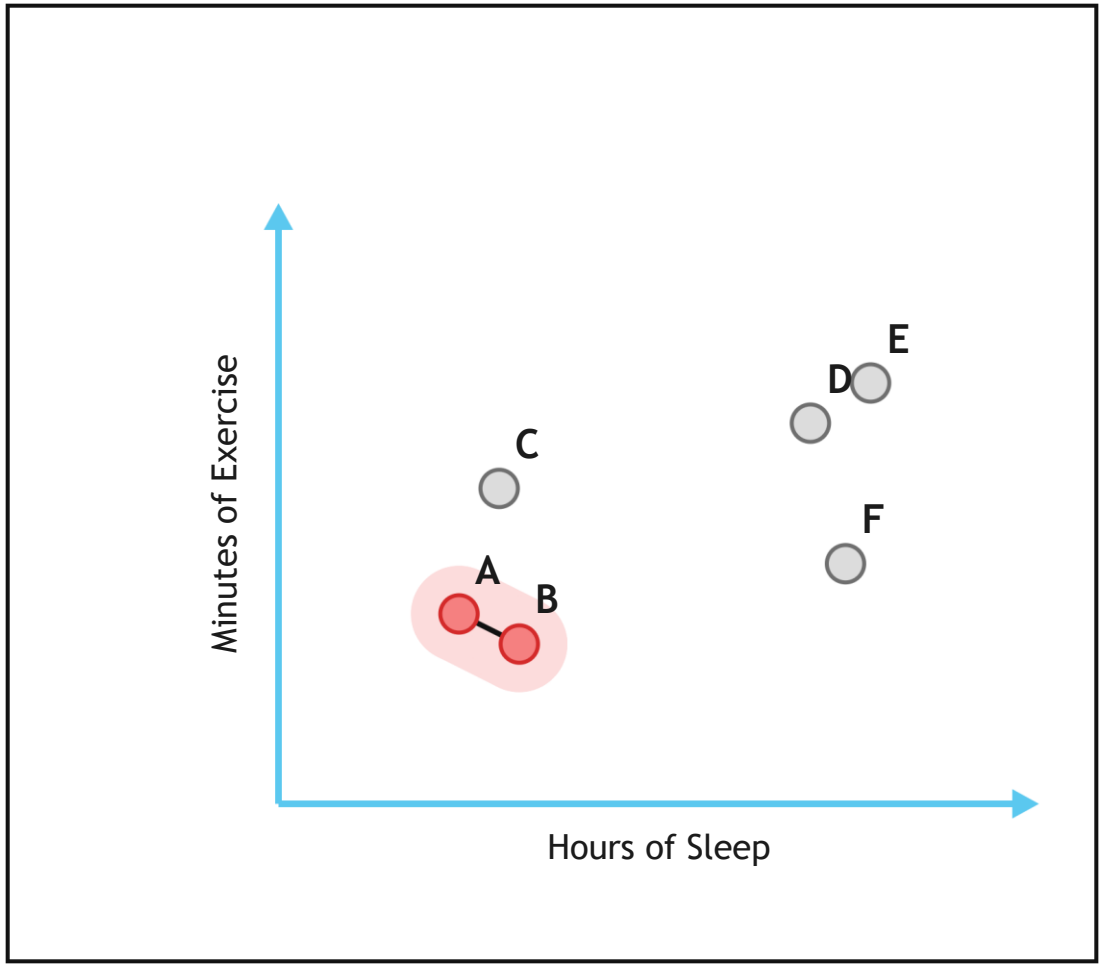


$C_2 \cup C_5$

	C_1		C_3	C_4
C_1	0	?	d'_{13}	d'_{14}
$C_2 \cup C_5$	?	0	?	?
C_3	d'_{31}	?	0	
C_4	d'_{41}	?		0

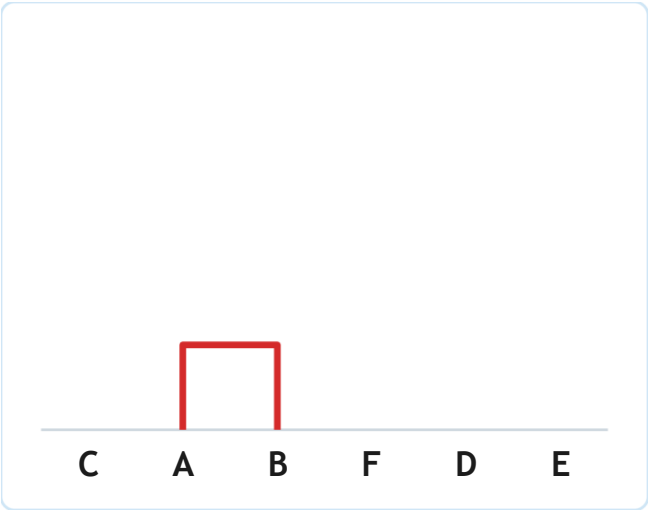
Agglomerative in Action

Six points, single link: merge the closest pair each step



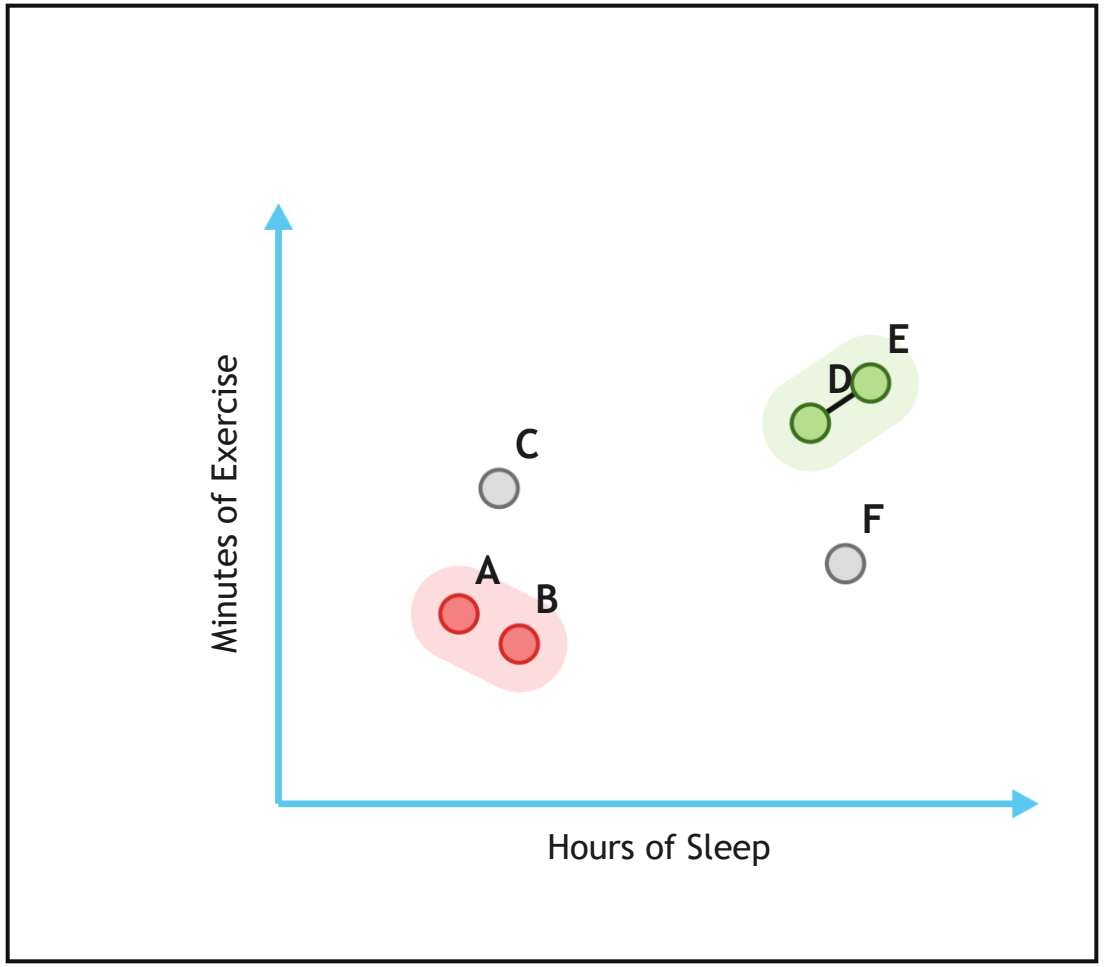
MERGE 1 OF 5

A and B are the closest pair, so they merge first. 5 clusters.



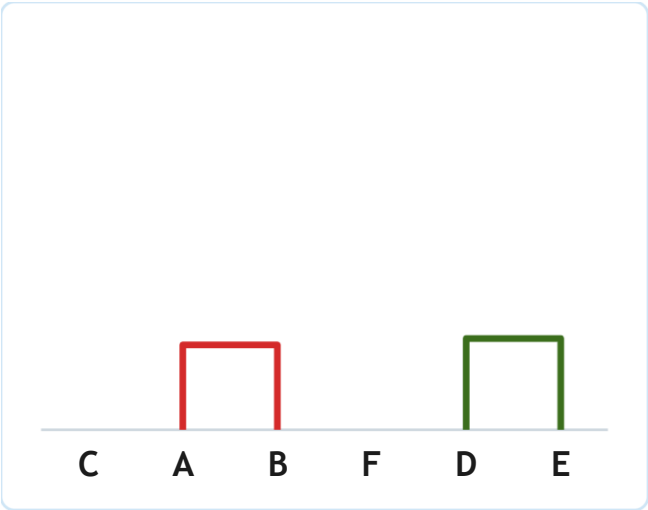
Agglomerative in Action

Six points, single link: merge the closest pair each step



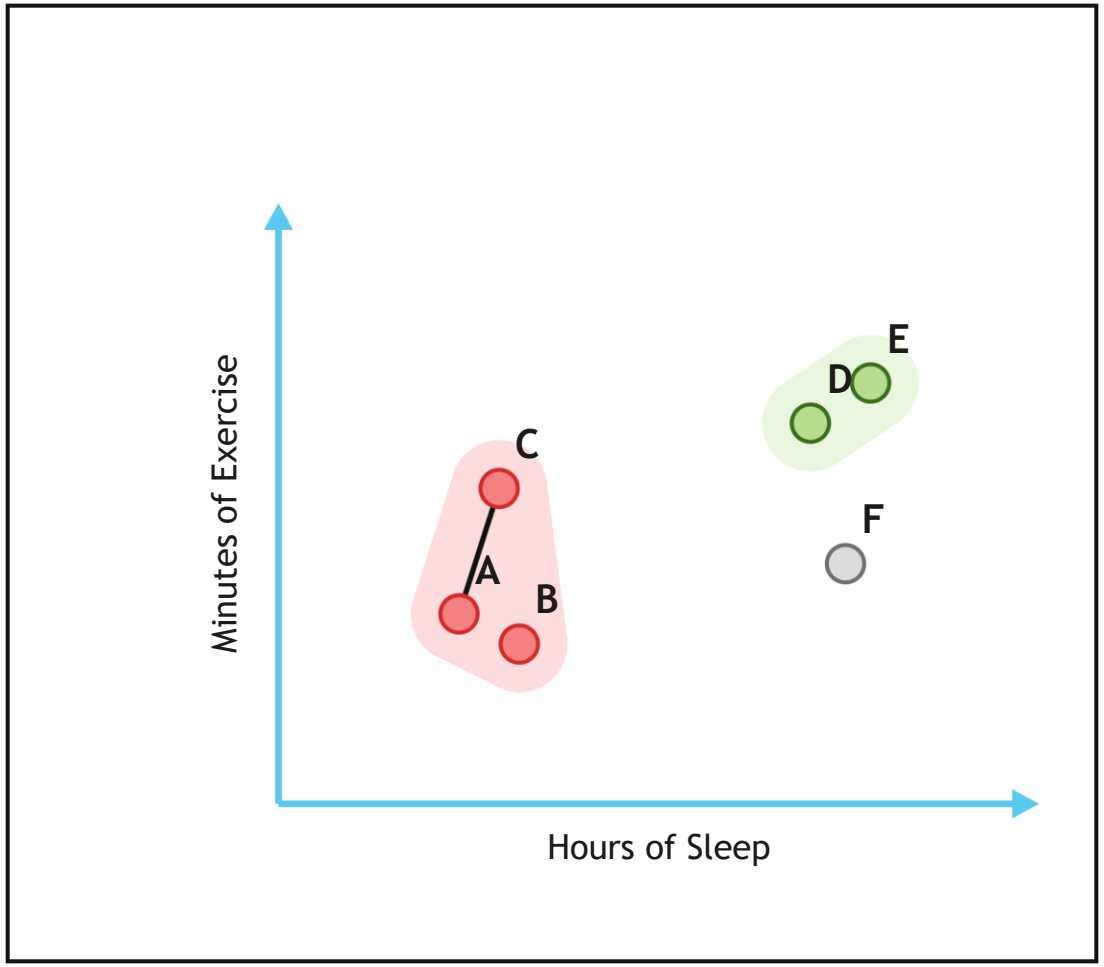
MERGE 2 OF 5

D and E are the next closest pair. 4 clusters.



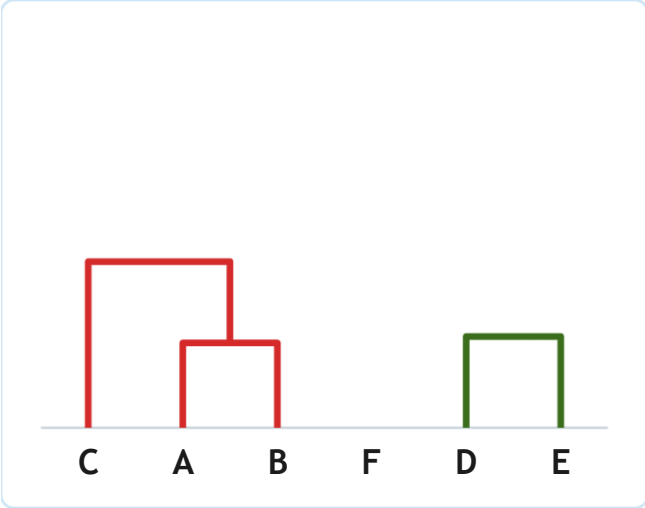
Agglomerative in Action

Six points, single link: merge the closest pair each step



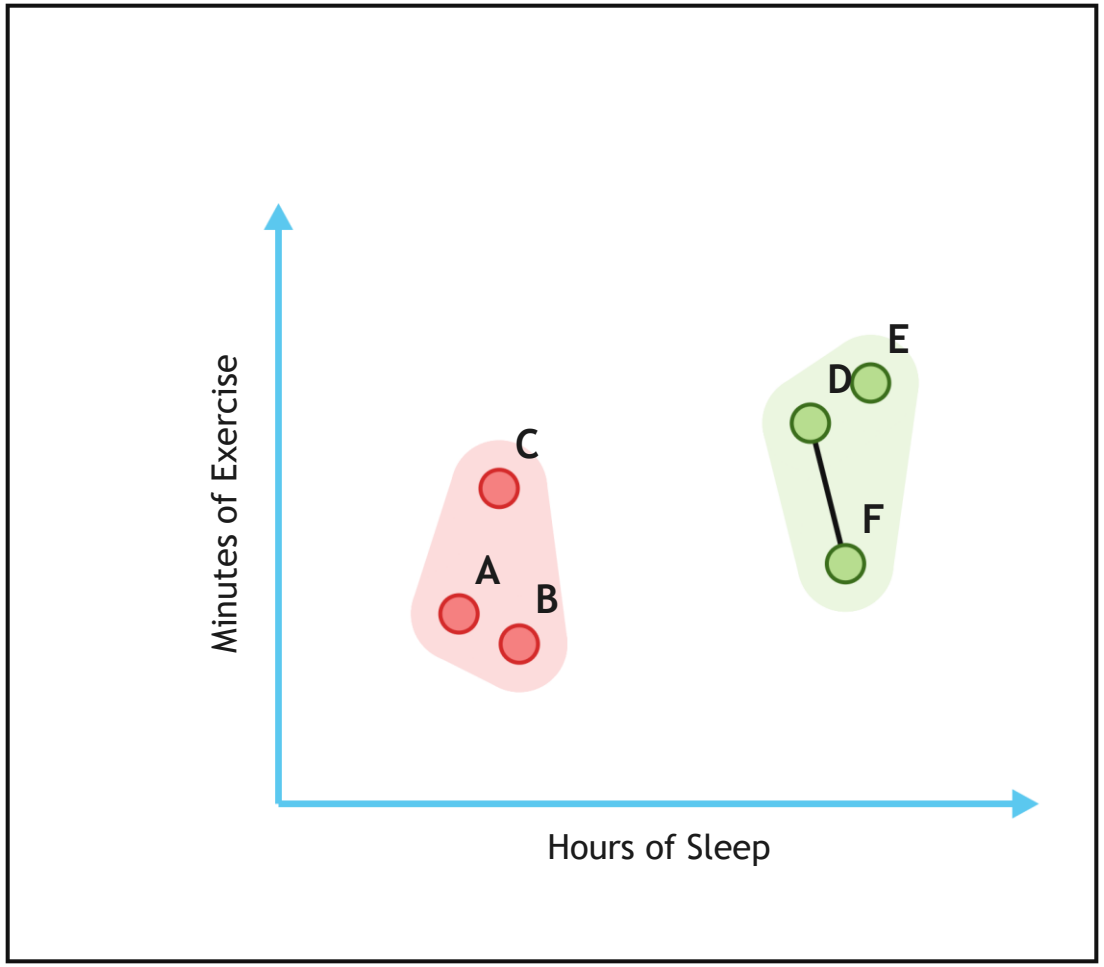
MERGE 3 OF 5

C is closest to {A, B}: its nearest member B is closer than any other pair. 3 clusters.



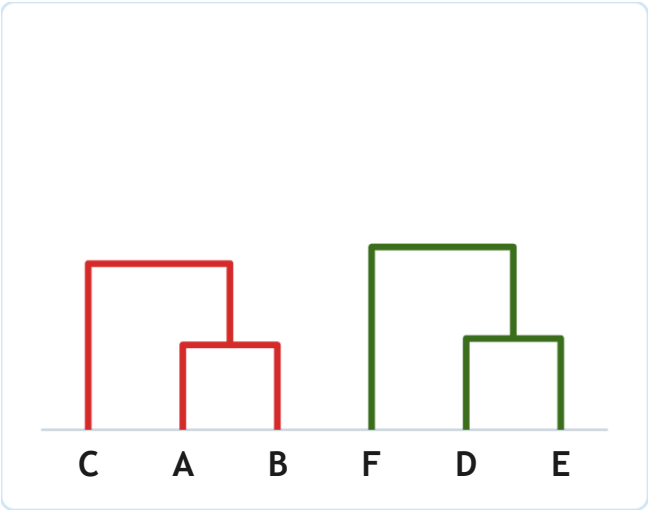
Agglomerative in Action

Six points, single link: merge the closest pair each step



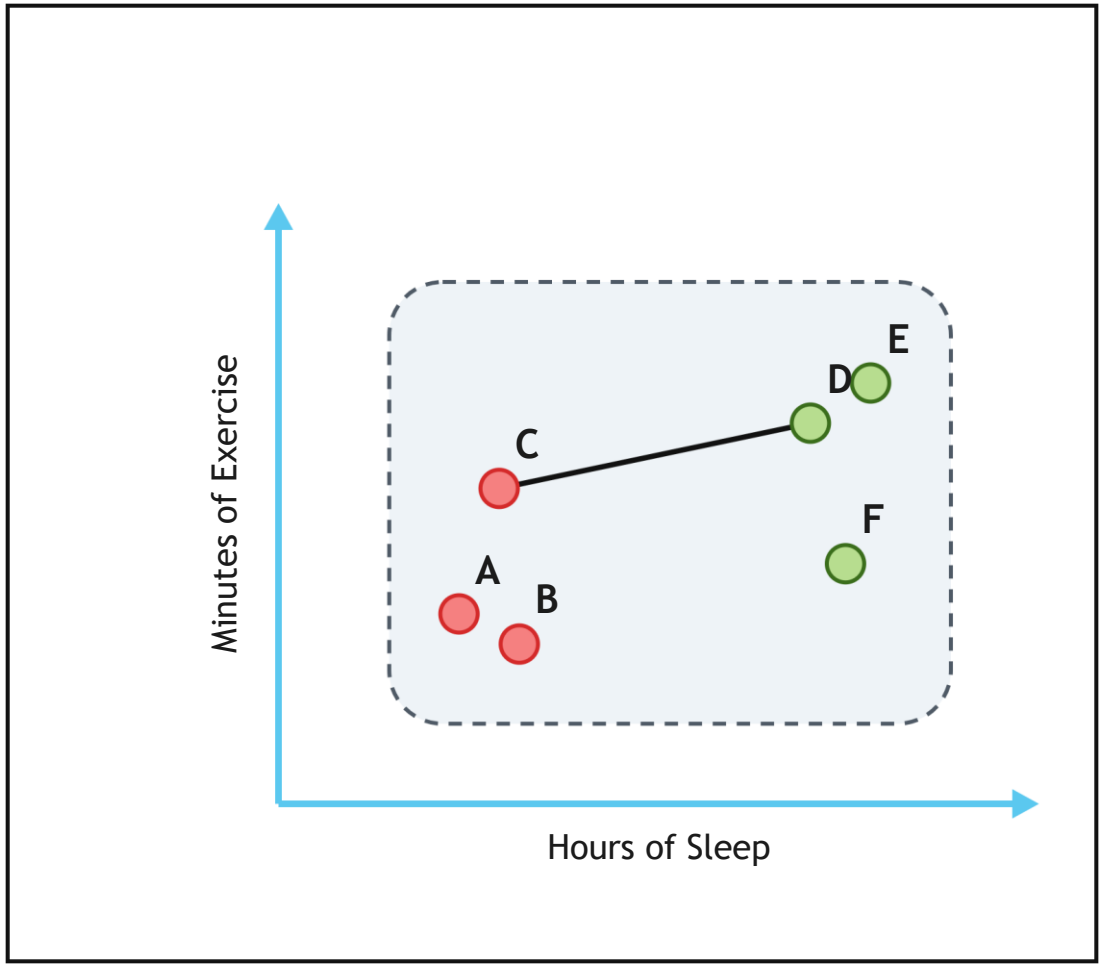
MERGE 4 OF 5

F joins {D, E} the same way. 2 clusters.



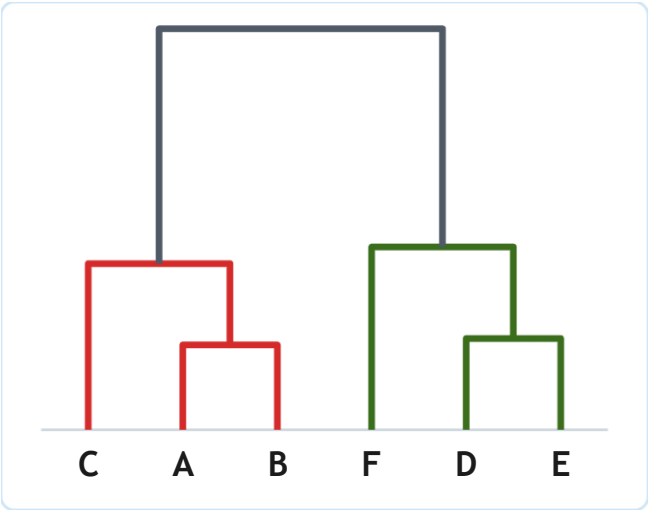
Agglomerative in Action

Six points, single link: merge the closest pair each step



MERGE 5 OF 5

The two remaining clusters merge at a large distance. 1 cluster; the tree is complete.



Agglomerative: Strengths and Weaknesses

Rich structure, but costly for large data

Strengths

- No need to choose k in advance
- Dendrogram shows structure at every level
- Works with any distance and many linkages
- Deterministic: same data, same tree

Weaknesses

- Merges are final: early mistakes cannot be undone
- Needs the $n \times n$ distance matrix: $O(n^2)$ memory, at least $O(n^2)$ time
- Single link chains; outliers can distort the tree
- Result depends on the linkage chosen

Part 2 of 2

Top-Down

Divisive clustering with recursive k-means

Divisive Clustering: The Idea

Start with everything together, then split

Ways to split just 32 objects into two non-empty groups:

2,147,483,647

In general $2^{n-1} - 1$. Trying every split is hopeless.

Use k-means, k = 2

Let 2-means find a good split quickly, then split the halves the same way.

Recursive k-means

Also called bisecting k-means (Steinbach, Karypis & Kumar, 2000).

In practice

Available as BisectingKMeans in scikit-learn.

Recursive (Bisecting) k-Means

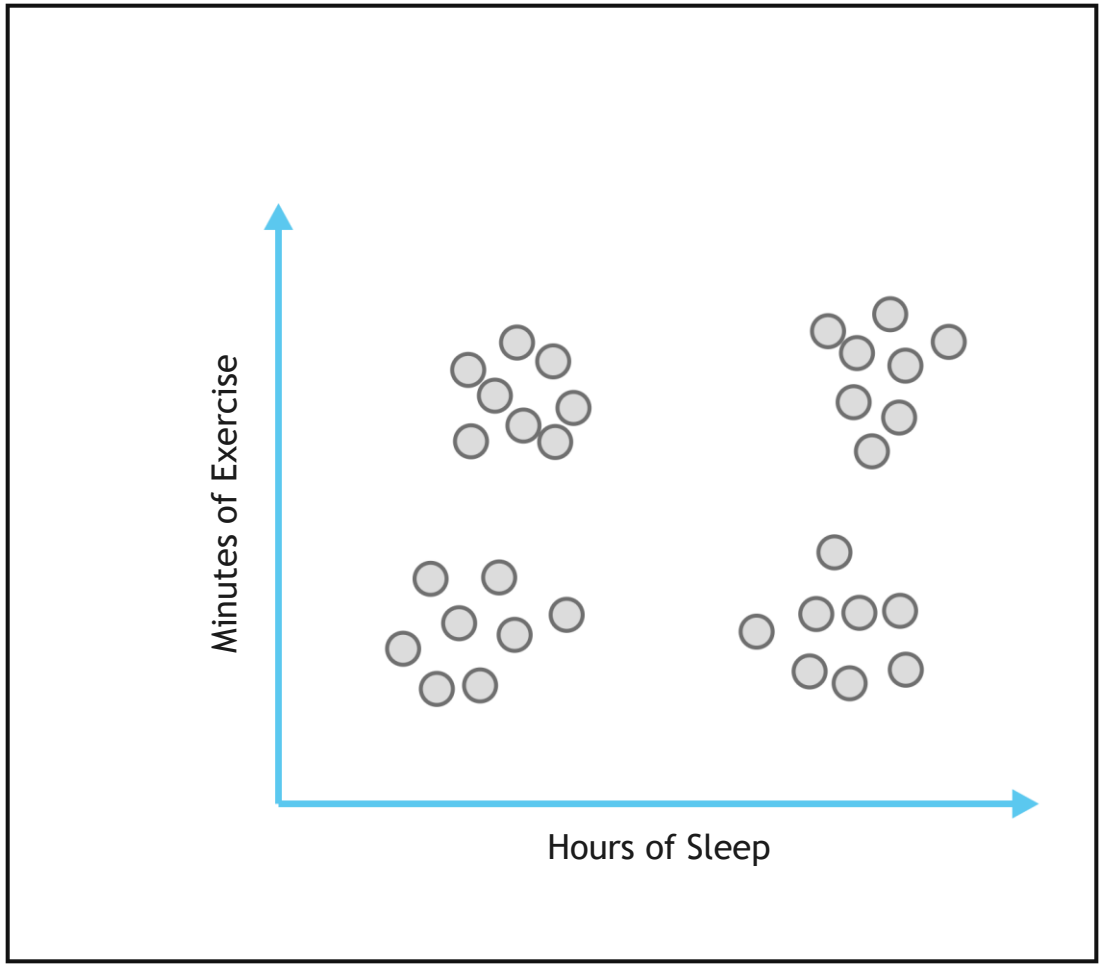
Split one cluster at a time with 2-means

1	**Start**	Put all objects in one cluster.
2	**Choose**	Pick the cluster to split: the one with the largest SSE.
3	**Bisect**	Split it with k-means, k = 2. Try several starts; keep the lowest SSE.
4	**Replace**	Replace the cluster with its two halves.
5	**Repeat**	Repeat steps 2–4 until there are k clusters.

Alternatives: split the largest cluster, or stop when every cluster is small or tight enough.

Recursive k-Means in Action

Splitting 32 people into 4 groups, one 2-means split at a time



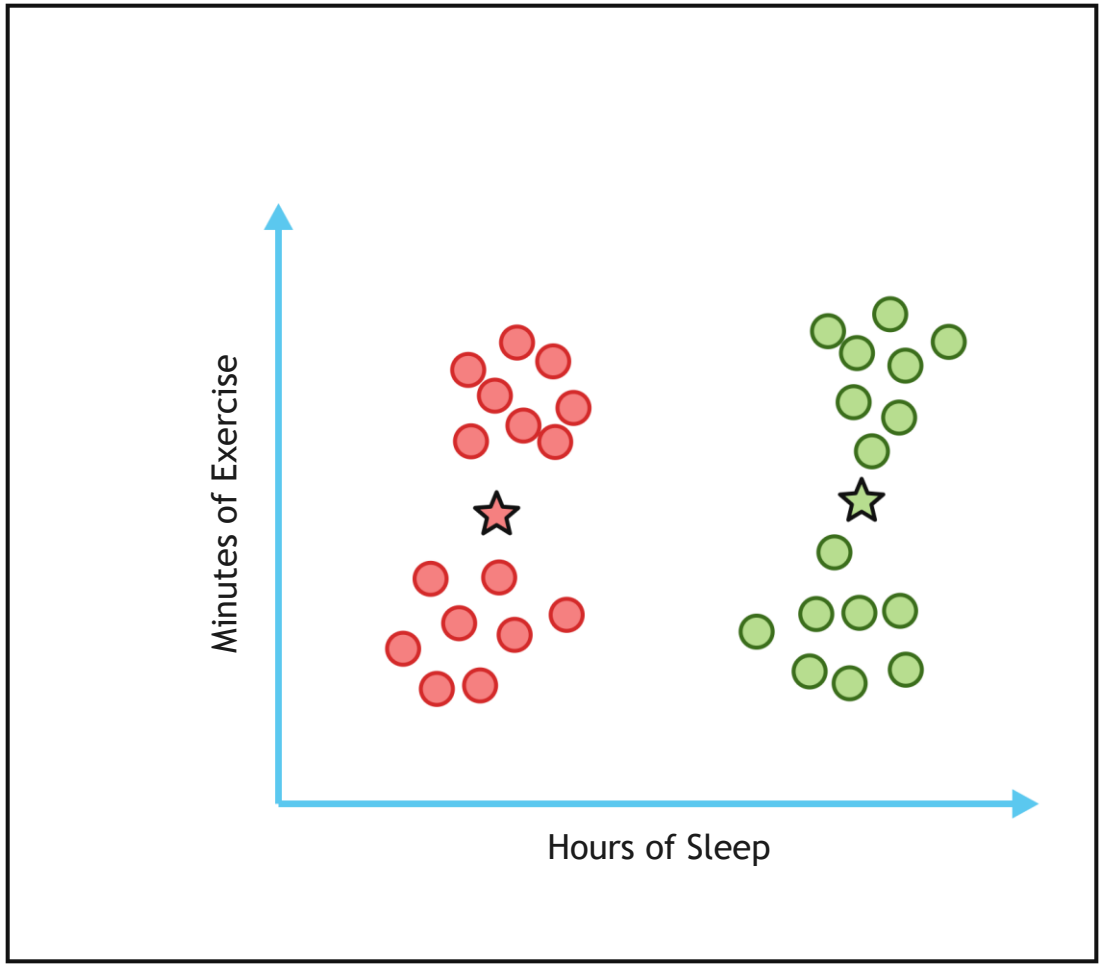
START

All 32 people are in one cluster.

All · 32

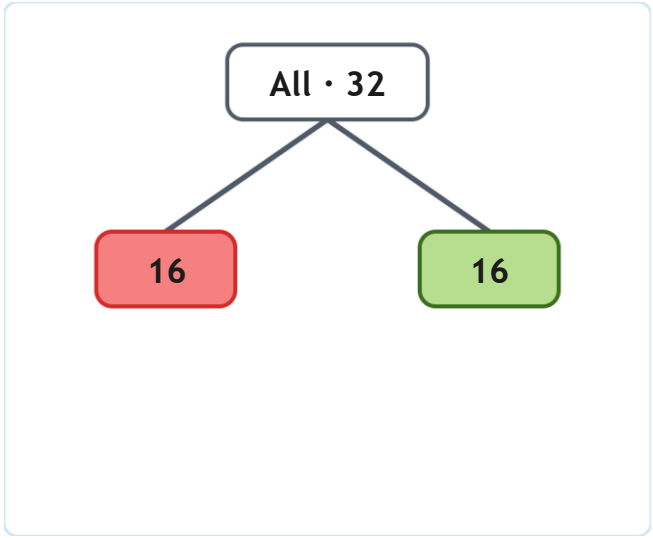
Recursive k-Means in Action

Splitting 32 people into 4 groups, one 2-means split at a time



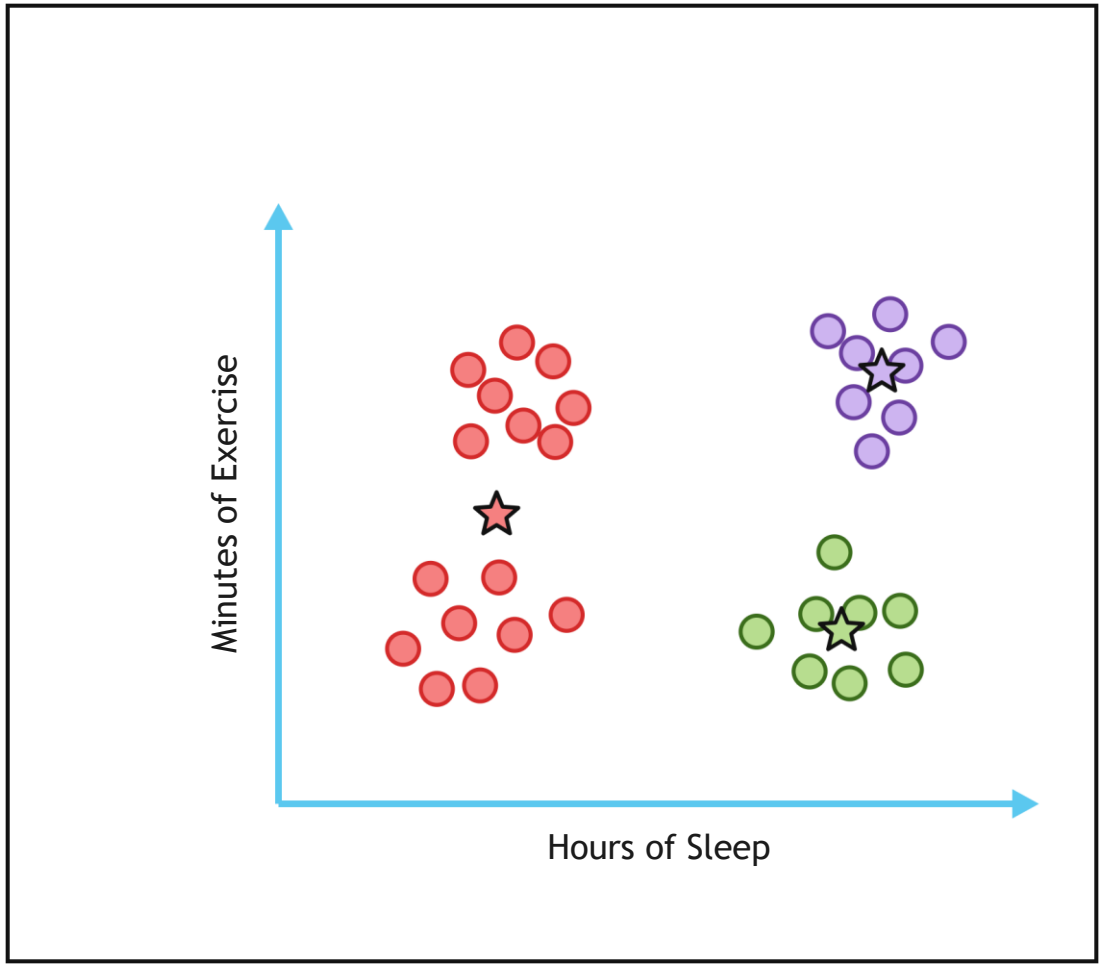
SPLIT 1 OF 3

2-means on everyone: less sleep (left) vs more sleep (right).



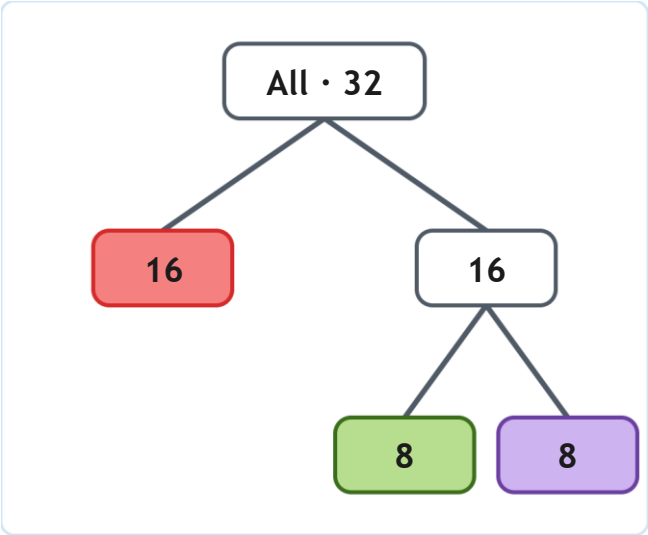
Recursive k-Means in Action

Splitting 32 people into 4 groups, one 2-means split at a time



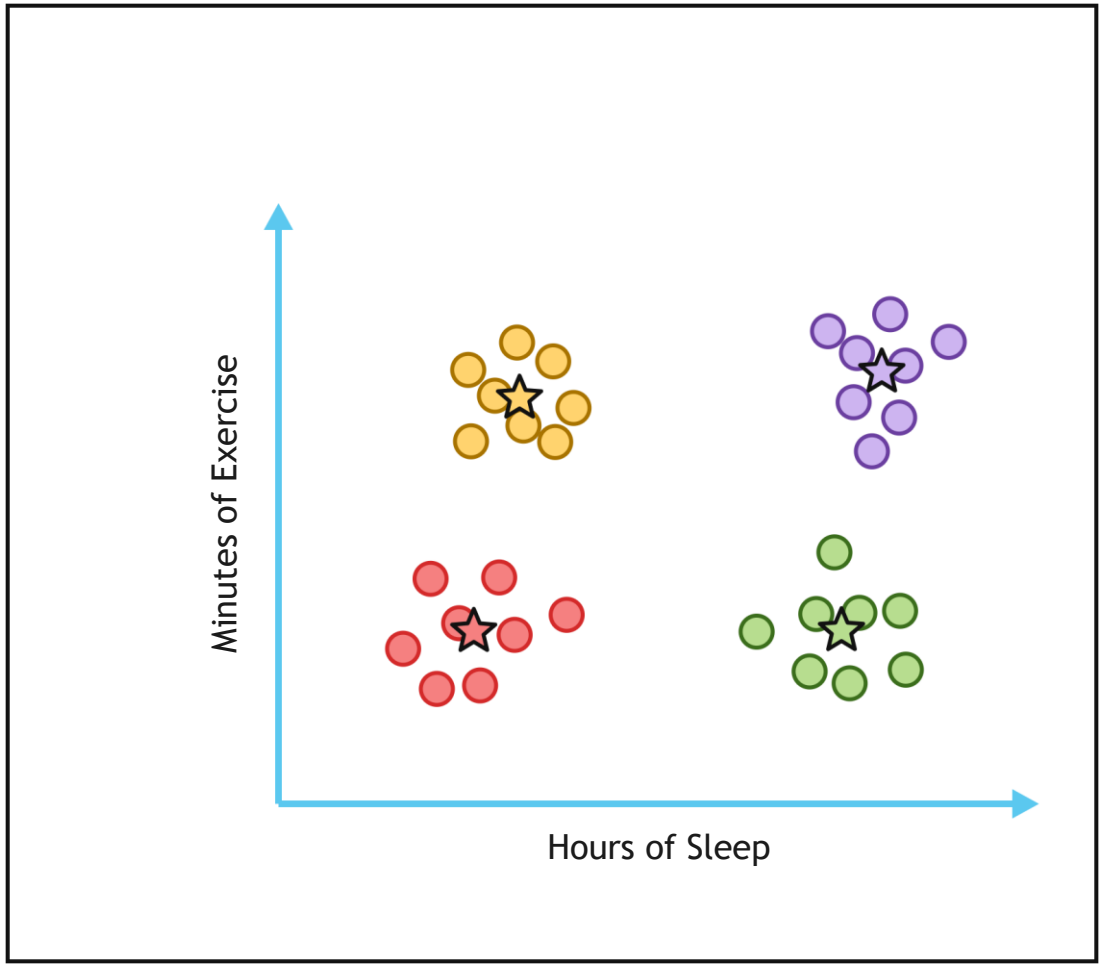
SPLIT 2 OF 3

The right cluster has the larger SSE (1.31 vs 1.11), so split it next: more vs less exercise.



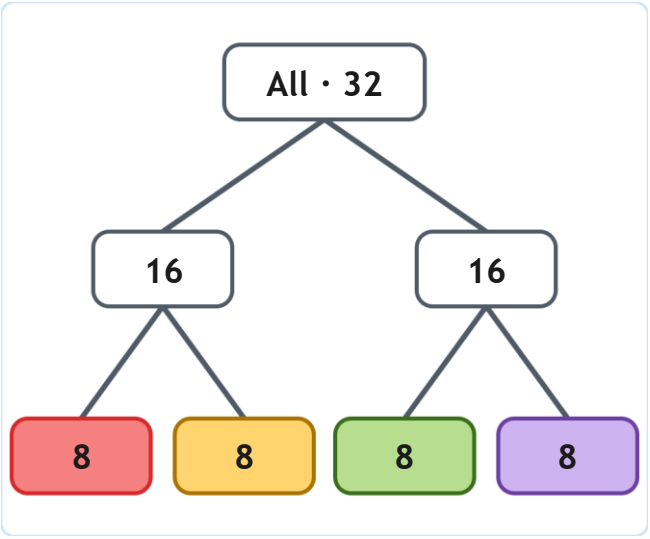
Recursive k-Means in Action

Splitting 32 people into 4 groups, one 2-means split at a time



SPLIT 3 OF 3

Now the left cluster has the largest SSE, so split it too. 4 clusters: stop here, or keep splitting.



Recursive k-Means: Strengths and Weaknesses

Scales well, but every split inherits k-means' limits

Strengths

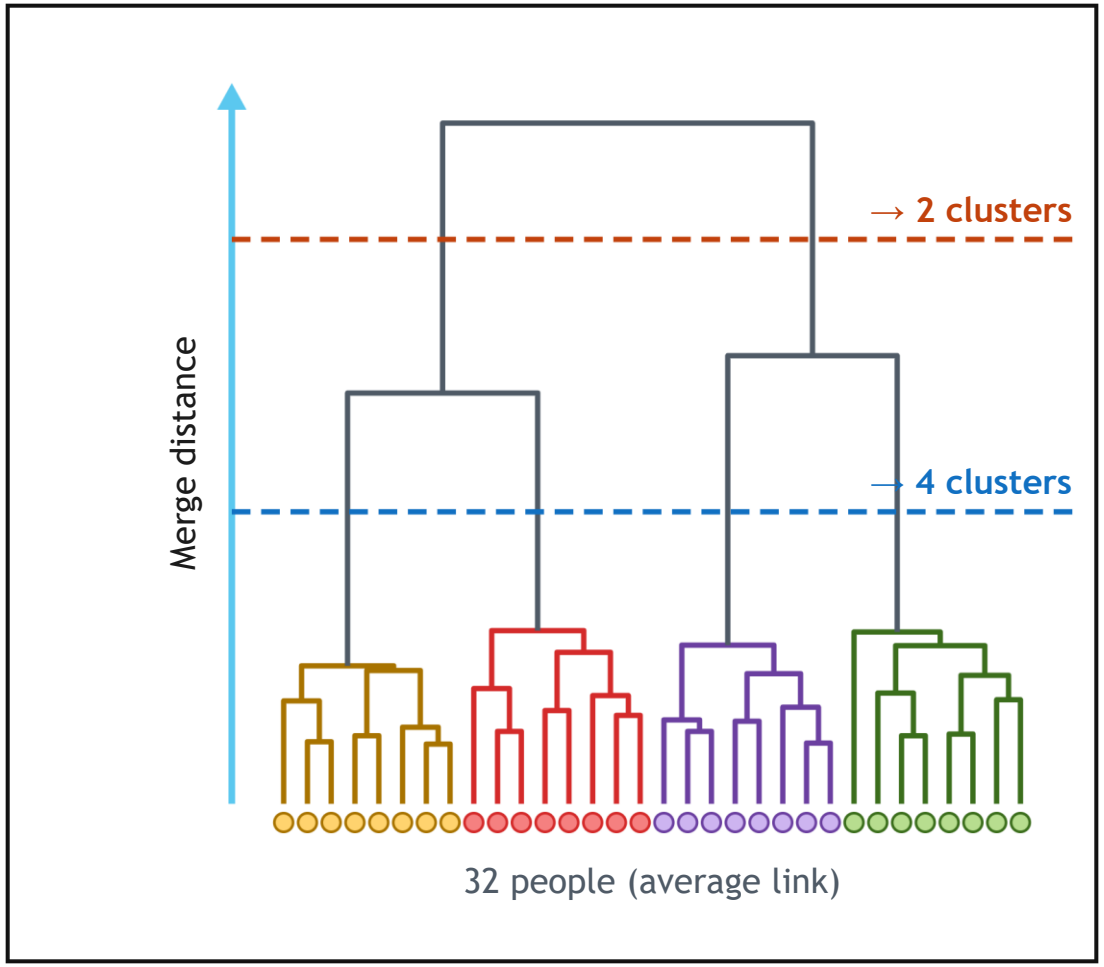
- Fast: each step is one 2-means run, so it scales to large data
- Sees global structure first: top splits use all the data
- Stop as soon as you have enough clusters
- Tends to give more evenly sized clusters

Weaknesses

- Splits are final: a bad early split cannot be undone
- Inherits k-means limits: numeric data, round clusters
- Each split depends on random starts
- Needs rules for which cluster to split and when to stop

Choosing the Number of Clusters

Cut the dendrogram where the branches are long



Cut the tree

A horizontal cut gives a flat clustering with as many clusters as branches crossed.

Where to cut

Cut across a long stretch with no merges: those merges join very different groups.

Nested

The 4 clusters split the 2 clusters, with no need to re-run.

Side-by-Side Comparison

Bottom-up agglomerative vs top-down recursive k-means

	Agglomerative	Recursive k-means
Direction	Bottom-up	Top-down
Starts with	n single-object clusters	One cluster with all objects
Each step	Merge the two closest clusters	Split one cluster with 2-means
Key choice	Linkage: single, complete, average, Ward	Which cluster to split; when to stop
Data	Any distance matrix	Numeric (needs means)
Cost	$O(n^2)$ memory, at least $O(n^2)$ time	One 2-means run per split: scales well
Undo?	No: merges are final	No: splits are final
Strongest at	Small data, fine local detail	Large data, big top-level groups

Key Takeaways

Two directions, one tree

1 Hierarchy

A tree of nested clusters. Cut the dendrogram to choose the number of clusters.

2 Bottom-up

Agglomerative: merge the two closest clusters. Linkage matters; costs $O(n^2)$.

3 Top-down

Recursive k-means: split the loosest cluster with 2-means. Fast, k-means limits.

Both directions build a dendrogram; neither can undo a step. Choose by data size and by which levels of the tree matter most.